

NumPy.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

RAM Disk

Share

The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy.

Arrays are very frequently used in data science, where speed and resources are very important.

Installation of NumPy

```
pip install numpy
```

Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)

Import NumPy

Once NumPy is installed, import it in your applications by adding the import keyword:

```
import numpy

arr = numpy.array([1, 2, 3, 4, 5])

print(arr)
```

[1 2 3 4 5]

NumPy as np

NumPy is usually imported under the np alias.

alias: In Python alias are an alternate name for referring to the same thing.

Create an alias with the as keyword while importing:

```
import numpy as np
```

Now the NumPy package can be referred to as np instead of numpy.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

[1 2 3 4 5]

NumPy Creating Arrays

Create a NumPy ndarray Object

NumPy is used to work with arrays. The array object in NumPy is called ndarray.

We can create a NumPy ndarray object by using the array() function.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

[1 2 3 4 5]

NumPy Creating Arrays

Create a NumPy ndarray Object

NumPy is used to work with arrays. The array object in NumPy is called ndarray.

We can create a NumPy ndarray object by using the array() function.

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])

print(arr)

print(type(arr))
```

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
```

Dimensions in Arrays

A dimension in arrays is one level of array depth (nested arrays).

nested array: are arrays that have arrays as their elements.

0-D Arrays

0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

Example

Create a 0-D array with value 42

```
import numpy as np

arr = np.array(42)

print(arr)
```

```
42
```

1-D Arrays

An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array.

These are the most common and basic arrays.

Example

Create a 1-D array containing the values 1,2,3,4,5:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

```
[1 2 3 4 5]
```

2-D Arrays

An array that has 1-D arrays as its elements is called a 2-D array.

These are often used to represent matrix or 2nd order tensors.

NumPy has a whole sub module dedicated towards matrix operations called `numpy.mat`

Example

Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr)
```

```
[[1 2 3]
 [4 5 6]]
```

3-D arrays

An array that has 2-D arrays (matrices) as its elements is called 3-D array.

These are often used to represent a 3rd order tensor.

Example

Create a 3-D array with two 2-D arrays, both containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]])

print(arr)
```

```
[[[1 2 3]
  [4 5 6]]

 [[1 2 3]
  [4 5 6]]]
```

Check Number of Dimensions?

NumPy Arrays provides the `ndim` attribute that returns an integer that tells us how many dimensions the array have.

Example

Check how many dimensions the arrays have:

```
import numpy as np

a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(a.ndim)
print(b.ndim)
print(c.ndim)
print(d.ndim)
```

```
0
1
2
3
```

Higher Dimensional Arrays

An array can have any number of dimensions.

When the array is created, you can define the number of dimensions by using the `ndmin` argument.

Example

Create an array with 5 dimensions and verify that it has 5 dimensions:

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)
print('number of dimensions :', arr.ndim)
```

```
[[[[[1 2 3 4]]]]]
number of dimensions : 5
```

In this array the innermost dimension (5th dim) has 4 elements, the 4th dim has 1 element that is the vector, the 3rd dim has 1 element that is the matrix with the vector, the 2nd dim has 1 element that is 3D array and 1st dim has 1 element that is a 4D array.

NumPy Array Indexing

Access Array Elements

Array indexing is the same as accessing an array element.

We can access an array element by referring to its index number.

The indexes in NumPy arrays start with 0. meaning that the first element has index 0. and the

second has index 1 etc.

- Get the first element from the following array:

```
[1]
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[0])
```

1

- Example

Get the second element from the following array.

```
[1]
import numpy as np
arr = np.array([1, 2, 3, 4])

print(arr[1])
```

2

- Example

Get third and fourth elements from the following array and add them.

```
[1]
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[2] + arr[3])
```

7

Access 2-D Arrays

To access elements from 2-D arrays we can use comma separated integers representing the dimension and the index of the element.

Think of 2-D arrays like a table with rows and columns, where the dimension represents the row and the index represents the column.

- Practical: Indexing & Slicing in 2D NumPy Array

To understand how to access and extract elements from a 2D array (matrix) using indexing and slicing.

Create a 2D Array (Matrix)

Double-click (or enter) to edit

```
[1]
✓ Os
import numpy as np
mat = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

print(mat)
```

[[10 20 30]
 [40 50 60]
 [70 80 90]]

- Index positions:

	Col0	Col1	Col2
Row0 →	10	20	30
Row1 →	40	50	60
Row2 →	70	80	90

▼ PART A: 2D Indexing (Single Element)

Syntax:

`array[row_index, column_index]`

Double-click (or enter) to edit

```
[2]  
✓ 0s #Examples:  
print(mat[0, 0]) # 10  
print(mat[1, 2]) # 60  
print(mat[2, 1]) # 80  
print(mat[-1, -1]) # 90
```

```
10  
60  
80  
90
```

Meaning:

`mat[1,2]` → 2nd row, 3rd column

Question:

What is the value at row 0, column 2?

▼ PART B: Row Slicing

Extract complete rows

```
[3]  
✓ 0s print(mat[0, :]) # First row  
print(mat[1, :]) # Second row
```

```
[10 20 30]  
[40 50 60]
```

▼ Extract multiple rows

```
[4]  
✓ 0s print(mat[0:2, :])
```

```
[[10 20 30]  
 [40 50 60]]
```

means all columns

▼ PART C: Column Slicing

Extract complete columns

```
[5]  
✓ 0s print(mat[:, 0]) # First column  
print(mat[:, 2]) # Third column
```

```
[10 40 70]  
[30 60 90]
```

#: means all rows

▼ Boolean Indexing in 2D

Filter values greater than 50

```
[7]  
✓ 0s print(mat[mat > 50])
```

```
[60 70 80 90]
```

▼ Modify Values using Indexing

```
[8]  
✓ 0s mat[0, 1] = 99  
print(mat)
```

```
[[10 99 30]  
 [40 50 60]  
 [70 80 90]]
```

Operation	Code	Meaning
Single element	<code>mat[1,2]</code>	2nd row, 3rd column
Row	<code>mat[1,:]</code>	Entire row
Column	<code>mat[:,1]</code>	Entire column
Sub-matrix	<code>mat[0:2,1:3]</code>	Part of matrix

make Below Table

Column 0 → Math

Column 1 → Science

Column 2 → English

```
• [[78 85 90]
   [65 70 75]
   [88 92 95]
   [50 55 60]
   [95 98 99]]
```

Solve the Below Problem

Math marks of Student 1

Science marks of Student 3

English marks of Student 5

Find English mark of Student 2?

Find the Science mark of Student 4?

Row Slicing (Student-wise Marks)

Find the Marks of one student

```
[11] ✓ Os print(marks[1, :]) # All subjects of Student 2
[65 70 75]
```

Double-click (or enter) to edit

Marks of first three students

```
[12] print(marks[0:3, :])
[[78 85 90]
 [65 70 75]
 [88 92 95]]
```

Column Slicing (Subject-wise Marks)

Math marks of all students

```
[13] ✓ Os print(marks[:, 0])
[78 65 88 50 95]
```

Science marks of all students

```
[14] ✓ Os print(marks[:, 1])
... [85 70 92 55 98]
```

```
[9] # marks = np.array([
#     [78, 85, 90], # Student 1
#     [65, 70, 75], # Student 2
#     [88, 92, 95], # Student 3
#     [50, 55, 60], # Student 4
#     [95, 98, 99]  # Student 5
# ])
```

```
# print(marks)
```

```
[[78 85 90]
 [65 70 75]
 [88 92 95]
 [50 55 60]
 [95 98 99]]
```

Example

Access the element on the first row, second column:

```
import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('2nd element on 1st row: ', arr[0, 0])
```

2nd element on 1st row: 6

Double-click (or enter) to edit

Start coding or [generate](#) with AI.

```
[60 70 80 90]
```

Double-click (or enter) to edit

Example

Access the element on the 2nd row, 5th column:

```
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('5th element on 2nd row: ', arr[1, 4])
```

5th element on 2nd row: 10

Access 3-D Arrays

To access elements from 3-D arrays we can use comma separated integers representing the dimensions and the index of the element.

Example

Access the third element of the second array of the first array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]])

print(arr[0, 1, 2])
```

6

Example Explained

arr[0, 1, 2] prints the value 6.

And this is why:

The first number represents the first dimension, which contains two arrays:

```
[[1, 2, 3], [4, 5, 6]]
```

and:

```
[[7, 8, 9], [10, 11, 12]]
```

Since we selected 0, we are left with the first array:

```
[[1, 2, 3], [4, 5, 6]]
```

The second number represents the second dimension, which also contains two arrays:

```
[1, 2, 3]
```

and:

```
[4, 5, 6]
```

Since we selected 1, we are left with the second array:

[4, 5, 6]

The third number represents the third dimension, which contains three values:

4

5

6

Since we selected 2, we end up with the third value:

6

```
[ ] import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]])

print(arr[1, 0, 1])
```

8

Negative Indexing

Use negative indexing to access an array from the end.

Example

Print the last element from the 2nd dim:

```
[ ] import numpy as np

arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])

print('Last element from 2nd dim: ', arr[0, -1])
```

Last element from 2nd dim: 5

NumPy Array Slicing

Slice elements from index 1 to index 5 from the following array:

```
[ ] import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5])
```

[2 3 4 5]

Double-click (or enter) to edit

Slicing arrays

Slicing in python means taking elements from one given index to another given index.

We pass slice instead of index like this: [start:end].

We can also define the step, like this: [start:end:step].

If we don't pass start its considered 0

If we don't pass end its considered length of array in that dimension

If we don't pass step its considered 1

Double-click (or enter) to edit

Double-click (or enter) to edit

```
[ ] import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[1:5])
```

[2 3 4 5]

Note: The result includes the start index but excludes the end index

Note: The result includes the start index, but excludes the end index.

Example

Slice elements from index 4 to the end of the array:

```
[1] import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[4:])
```

[5 6 7]

Example

Slice elements from the beginning to index 4 (not included):

```
[1] import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[:4])
```

[1 2 3 4]

Negative Slicing

Use the minus operator to refer to an index from the end:

Example

Slice from the index 3 from the end to index 1 from the end:

```
[1] import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[-3:-1])
```

[5 6]

[1] Start coding or [generate](#) with AI.

STEP

Use the step value to determine the step of the slicing:

Example

Return every other element from index 1 to index 5:

Double-click (or enter) to edit

```
[1] import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[1:5:2])
```

[2 4]

arr[1:4] → elements from index 1 to 3

arr[:3] → first three elements

arr[2:] → elements from index 2 to end

arr[::2] → alternate elements starting from first

arr[1::2] → alternate elements starting from second

Colab paid products - [Cancel contracts here](#)

 Variables  Terminal



✓ 7:57 AM  Python 3